

OSSP-PRACTICAL-WEEK5

Session5

Task1: Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

Source code:

```
GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <time.h>

#define COUNT 100000

int main() {
    int pipefd[2];
    pid_t pid;
    int data;
    long long sum = 0;

    struct timespec start, end;

    if (pipe(pipefd) == -1) {
        perror("pipe");
        return 1;
    }

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid > 0) {
        /* Parent - Producer */
        close(pipefd[0]);

        clock_gettime(CLOCK_MONOTONIC, &start);

        for (int i = 1; i <= COUNT; i++) {
            data = i;

            if (write(pipefd[1], &data, sizeof(data)) == -1) {
                perror("write");
                close(pipefd[1]);
                return 1;
            }
        }

        close(pipefd[1]);

        wait(NULL);

        clock_gettime(CLOCK_MONOTONIC, &end);

        double elapsed =
            (end.tv_sec - start.tv_sec) +
            (end.tv_nsec - start.tv_nsec) / 1e9;

        printf("\nProducer: Generated %d integers.\n", COUNT);
    }
}
```

```

GNU nano 7.2                                     practic:
printf("\nProducer: Generated %d integers.\n", COUNT);
printf("Communication time: %.6f seconds\n", elapsed);
printf("Communication rate: %.2f integers/second\n",
      COUNT / elapsed);
}
else {
    /* Child - Consumer */
    close(pipefd[1]);

    for (int i = 1; i <= COUNT; i++) {
        if (read(pipefd[0], &data, sizeof(data)) == -1) {
            perror("read");
            close(pipefd[0]);
            return 1;
        }

        sum += data;
    }

    close(pipefd[0]);

    printf("Consumer: Received %d integers.\n", COUNT);
    printf("Consumer: Sum = %lld\n", sum);
}

return 0;
}

```

Output:

```

root@LAPTOP-1COJKKG6:~# gcc practical_week5_task1.c
root@LAPTOP-1COJKKG6:~# ./practical_week5_task1
Consumer: Received 100000 integers.
Consumer: Sum = 5000050000

Producer: Generated 100000 integers.
Communication time: 0.034275 seconds
Communication rate: 2917620.80 integers/second
root@LAPTOP-1COJKKG6:~# |

```

Observation: I learned how to implement communication between a parent and child process using an anonymous pipe. I understood how the parent process acts as the producer by writing data into the pipe, while the child process acts as the consumer by reading the data. I also learned how to measure communication time and calculate the communication rate to evaluate the efficiency of inter-process communication.

Task2: Implement a C program to demonstrate inter-process communication using pipes, where the output of one process is connected to the input of another process, similar to the shell pipeline `ls -l | grep ".c"`.

Source code:

```

GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    int pipefd[2];
    pid_t pid1, pid2;

    if (pipe(pipefd) == -1) {
        perror("pipe");
        return 1;
    }

    /* First child: executes ls -l */
    pid1 = fork();

    if (pid1 < 0) {
        perror("fork");
        return 1;
    }

    if (pid1 == 0) {
        close(pipefd[0]);

        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);

```

```

GNU nano 7.2
    execlp("ls", "ls", "-l", NULL);

    perror("execlp ls");
    exit(1);
}

/* Second child: executes grep ".c" */
pid2 = fork();

if (pid2 < 0) {
    perror("fork");
    return 1;
}

if (pid2 == 0) {
    close(pipefd[1]);

    dup2(pipefd[0], STDIN_FILENO);
    close(pipefd[0]);

    execlp("grep", "grep", ".c", NULL);

    perror("execlp grep");
    exit(1);
}

/* Parent closes both pipe ends */
close(pipefd[0]);

```

```

/* Parent closes both pipe ends */
close(pipefd[0]);
close(pipefd[1]);

waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

printf("Pipeline execution completed.\n");

return 0;
}

```

Output:

```

root@LAPTOP-1C0JKKG6:~# ./practical_week5_task2
-rwxr-xr-x 1 root root 16368 Aug 22 03:03 practical_week4_task1
-rw-r--r-- 1 root root 1189 Aug 22 03:06 practical_week4_task1.c
-rwxr-xr-x 1 root root 16232 Aug 22 03:27 practical_week4_task2
-rw-r--r-- 1 root root 570 Aug 22 03:27 practical_week4_task2.c
-rwxr-xr-x 1 root root 16368 Aug 22 03:48 practical_week5_task1
-rw-r--r-- 1 root root 1692 Aug 22 03:48 practical_week5_task1.c
-rwxr-xr-x 1 root root 16360 Aug 23 03:12 practical_week5_task2
-rw-r--r-- 1 root root 1103 Aug 23 03:12 practical_week5_task2.c
-rw-r--r-- 1 root root 559 Aug 11 10:15 session2_task1.c
-rw-r--r-- 1 root root 537 Aug 11 14:39 session2_task2.c
-rw-r--r-- 1 root root 4230 Aug 11 16:22 session3_task1.c
-rw-r--r-- 1 root root 2188 Aug 11 16:34 session3_task2.c
-rw-r--r-- 1 root root 2266 Aug 11 17:20 session4_task1.c
-rw-r--r-- 1 root root 2252 Aug 11 17:40 session4_task2.c
-rw-r--r-- 1 root root 1552 Aug 12 03:09 session5_task1.c
-rw-r--r-- 1 root root 3236 Aug 14 09:31 session5_task2.c
-rw-r--r-- 1 root root 1222 Aug 12 11:58 session6_task1.c
-rw-r--r-- 1 root root 1190 Aug 12 12:12 session6_task2.c
-rw-r--r-- 1 root root 1285 Aug 14 09:01 session7_task1.c
-rw-r--r-- 1 root root 1472 Aug 14 09:09 session7_task2.c
-rw-r--r-- 1 root root 2874 Aug 14 09:23 session8_task1.c
-rw-r--r-- 1 root root 2638 Aug 14 09:26 session8_task2.c
-rw-r--r-- 1 root root 879 Aug 11 09:58 skill_week1_task2.c
-rw-r--r-- 1 root root 53 Aug 8 06:03 source.txt
-rw-r--r-- 1 root root 1350 Aug 8 04:41 week1pt1.c
-rw-r--r-- 1 root root 1208 Aug 18 10:10 week2pt1.c
-rw-r--r-- 1 root root 1023 Aug 8 03:26 week3pt1.c
-rw-r--r-- 1 root root 376 Aug 8 03:32 week3pt2.c
Pipeline execution completed.
root@LAPTOP-1C0JKKG6:~#

```

Observation: I learned how to implement a shell-like pipeline using `fork()`, `pipe()`, `dup2()`, and `exec()`. I understood how the first child executes `ls -l` and sends its output through a pipe, while the second child receives the output and executes `grep ".c"`. I also learned how file descriptors can be redirected using `dup2()` and how the parent synchronizes both child processes using `waitpid()`.